



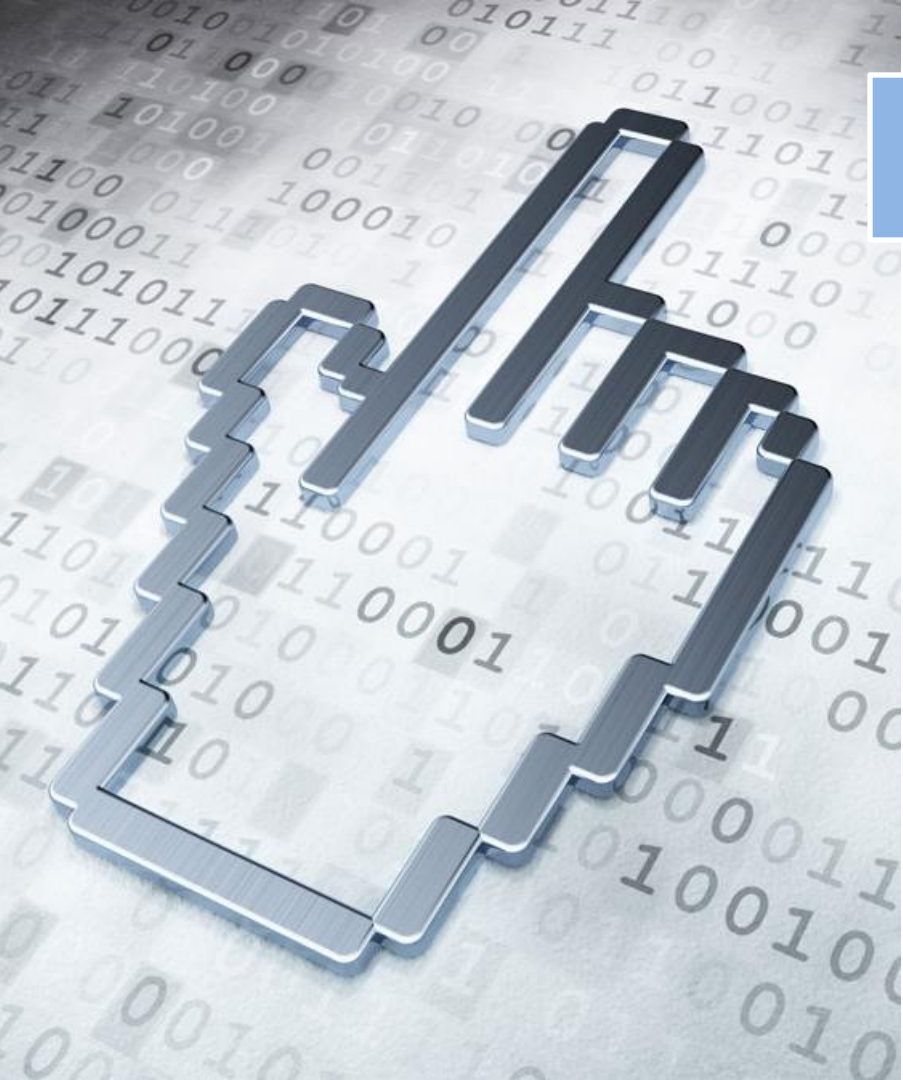
编译原理

第三章

词法分析

哈尔滨工业大学 陈鄞





本章内容

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

3.1 单词的描述

➤ 正则文法

例：描述标识符的正则文法

$$\textcircled{1} S \rightarrow aS' \mid bS' \mid \dots \mid zS'$$

$$\textcircled{2} S' \rightarrow aS' \mid bS' \mid \dots \mid zS' \mid \emptyset S' \mid 1S' \mid 2S' \mid \dots \mid 9S' \mid \varepsilon$$

➤ 正则表达式(Regular Expression, RE)

例

➤ 十进制整数的RE

$$(1|\dots|9)(0|\dots|9)^*|0$$

➤ 八进制整数的RE

$$0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$$

➤ 十六进制整数的RE

$$0x(0|1|\dots|9|a|\dots|f|A|\dots|F)(0|\dots|9|a|\dots|f|A|\dots|F)^*$$

是一种用来描述正则语言的更紧凑的表示方法

正则定义 (Regular Definition)

- 正则定义是具有如下形式的定义序列：

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

给一些 *RE* 命名，并在之后的 *RE* 中像使用字母表中的符号一样使用这些名字

其中：

- 每个 d_i 都是一个新符号，它们都不在字母表 Σ 中，而且各不相同
- 每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式

例 1

➤ *C*语言中标识符的正则定义

➤ $digit \rightarrow 0|1|2|\dots|9$

➤ $letter_ \rightarrow A|B|\dots|Z|a|b|\dots|z|_$

➤ $id \rightarrow letter_ (letter_|digit)^*$

例2

➤ (整型或浮点型) 无符号数的正则定义

➤ $digit \rightarrow 0|1|2|\dots|9$

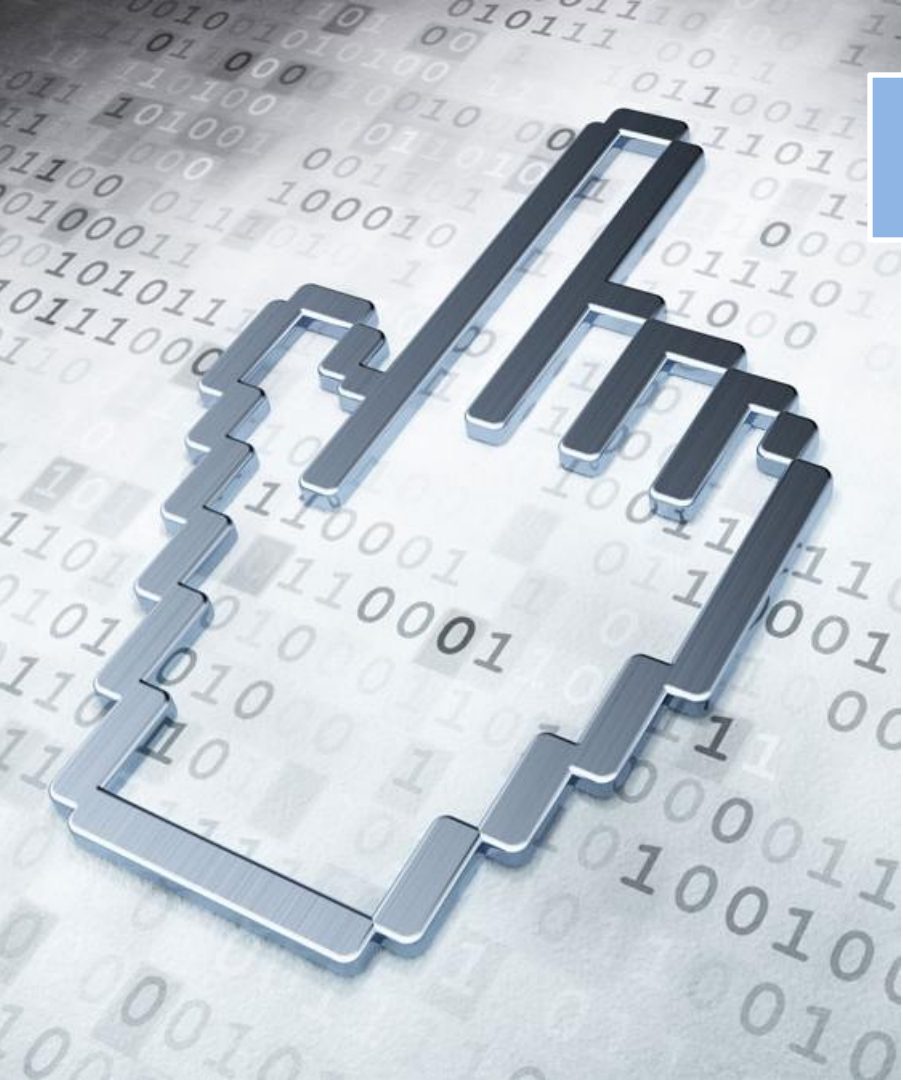
➤ $digits \rightarrow digit\ digit^*$

➤ $optionalFraction \rightarrow .digits|\epsilon$

➤ $optionalExponent \rightarrow (E(+|-|\epsilon)digits)|\epsilon$

➤ $number \rightarrow digits\ optionalFraction\ optionalExponent$

2	2.15	2.15E+3	2.15E-3	2.15E3	2E-3
---	------	---------	---------	--------	------



提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

3.2 单词的识别

- 3.2.1 有穷自动机 (*Finite Automata*)
- 3.2.2 有穷自动机的分类
- 3.2.3 从正则表达式到有穷自动机
- 3.2.4 识别单词的 *DFA*

3.2.1 有穷自动机

- 有穷自动机 (*Finite Automata*, *FA*) 由两位神经物理学家 *McCulloch* 和 *Pitts* 于 1948 年首先提出，是对一类处理系统建立的数学模型
- 这类系统具有一系列离散的输入输出信息和有穷数目的内部状态（状态：概括了对过去输入信息处理的状况）
- 系统只需要根据当前所处的状态和当前面临的输入信息就可以决定系统的后继行为。每当系统处理了当前的输入后，系统的内部状态也将发生改变

FA的典型例子

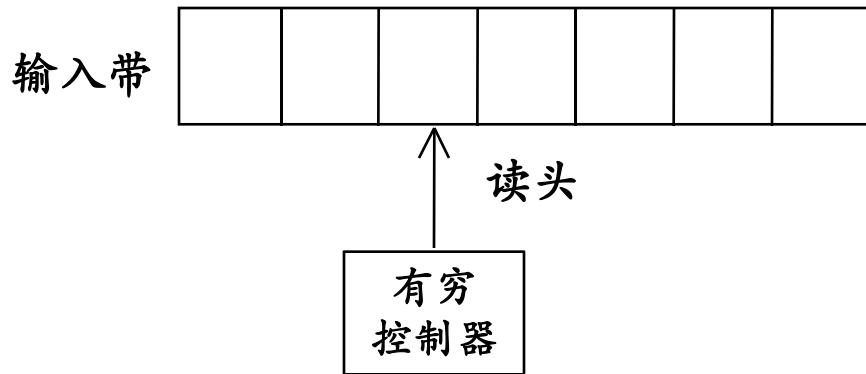
➤ 电梯控制装置

➤ 输入：顾客的乘梯需求（所要到达的层号）

➤ 状态：电梯所处的层数+运动方向

➤ 电梯控制装置并不需要记住先前全部的服务要求，只需要知道电梯当前所处的状态以及还没有满足的所有服务请求

FA模型



- **输入带** (*input tape*): 用来存放输入符号串
- **读头** (*head*): 从左向右逐个读取输入符号, 不能修改 (只读)、不能往返移动
- **有穷控制器** (*finite control*): 具有有穷个状态数, 根据当前的状态和当前输入符号控制转入下一状态

FA的表示

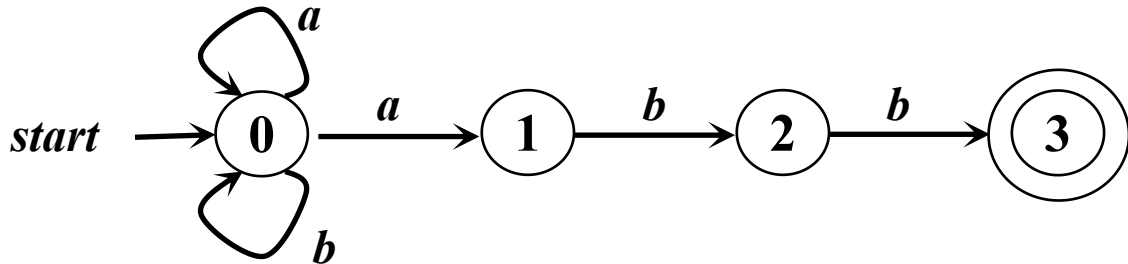
➤ 转换图 (Transition Graph)

➤ 结点：FA的状态

➤ 初始状态（开始状态）：只有一个，由start箭头指向

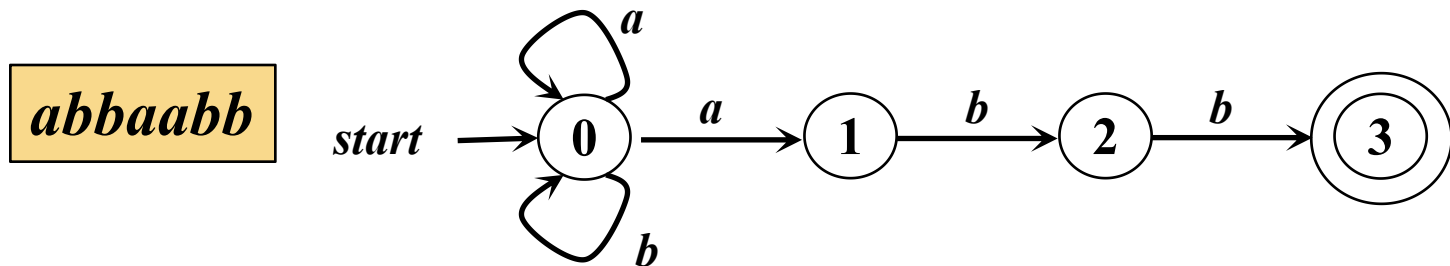
➤ 终止状态（接收状态）：可以有多个，用双圈表示

➤ 带标记的有向边：如果对于输入 a ，存在一个从状态 p 到状态 q 的转换，就在 p 、 q 之间画一条有向边，并标记上 a



FA定义 (接收) 的语言

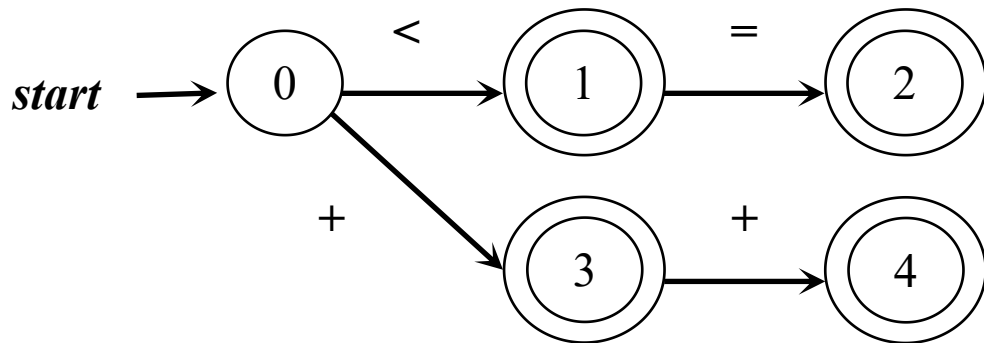
- 给定输入串 x ，如果存在一个对应于串 x 的从初始状态到某个终止状态的转换序列，则称串 x 被该FA接收
- 由一个有穷自动机 M 接收的所有串构成的集合称为是该FA定义 (或接收) 的语言，记为 $L(M)$



$L(M)$ = 所有以 abb 结尾的字母表 $\{a, b\}$ 上的串的集合

最长子串匹配原则 (Longest String Matching Principle)

- 当输入串的多个前缀与一个或多个模式匹配时，总是选择最长的前缀进行匹配



- 在到达某个终态之后，只要输入带上还有符号，*DFA*就继续前进，以便寻找尽可能长的匹配

3.2.2 *FA*的分类

- 确定的 *FA* (*Deterministic finite automata, DFA*)
- 非确定的 *FA* (*Nondeterministic finite automata, NFA*)

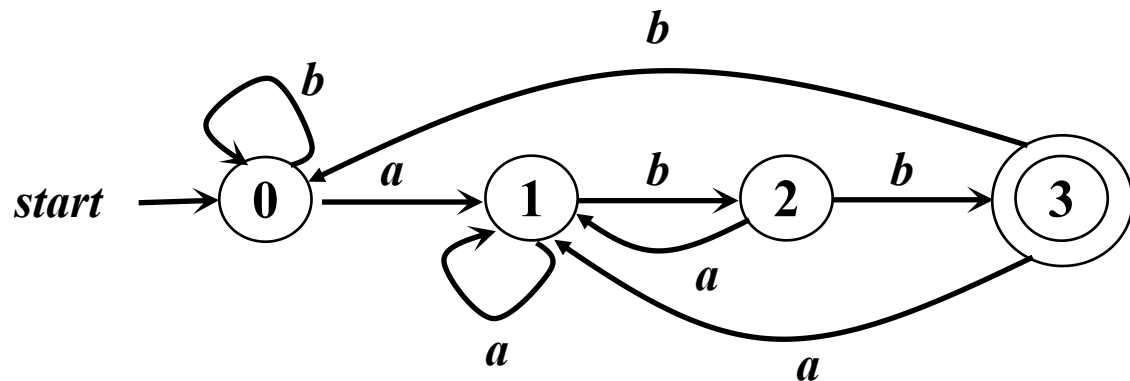
确定的有穷自动机 (DFA)

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入字母表, 即输入符号集合。假设 ε 不是 Σ 中的元素
- δ : 将 $S \times \Sigma$ 映射到 S 的转换函数。 $\forall s \in S, a \in \Sigma, \delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的状态
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

例：一个 DFA

$$M = (S, \Sigma, \delta, s_0, F)$$



转换表

状态 \ 输入	a	b
0	1	0
1	1	2
2	1	3
3 •	1	0

可以用转换表表示 DFA

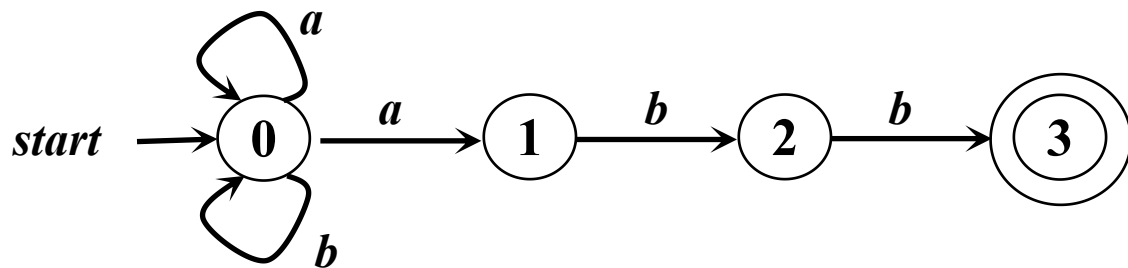
非确定的有穷自动机(NFA)

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入符号集合, 即输入字母表。假设 ε 不是 Σ 中的元素
- δ : 将 $S \times \Sigma$ 映射到 2^S 的转换函数。 $\forall s \in S, a \in \Sigma, \delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的状态集合
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

例：一个 *NFA*

$$M = (S, \Sigma, \delta, s_0, F)$$



转换表

状态 \ 输入	<i>a</i>	<i>b</i>
0	{0,1}	{0}
1	$\emptyset$	{2}
2	$\emptyset$	{3}
3 •	$\emptyset$	$\emptyset$

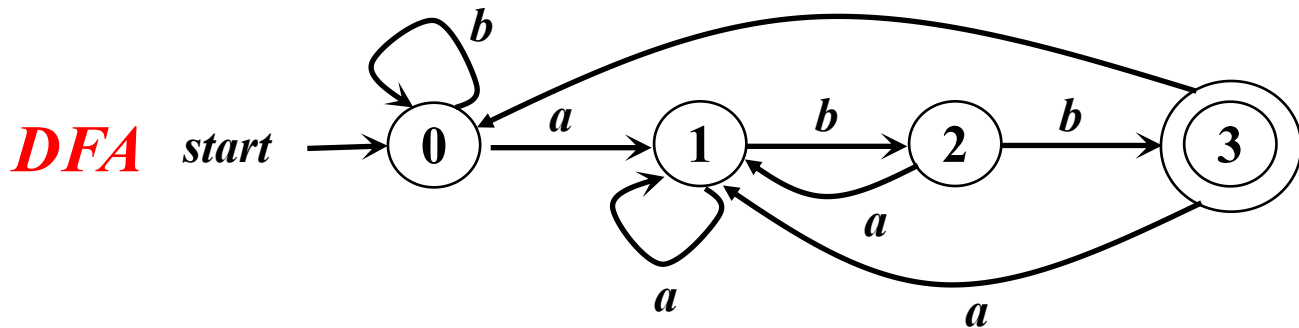
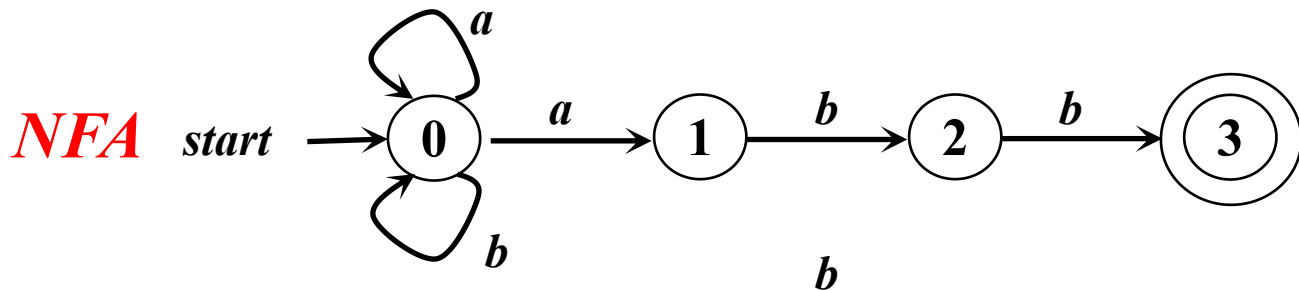
如果转换函数没有给出对应于某个状态-输入对的信息，就把 $\emptyset$ 放入相应的表项中

DFA 和 NFA 的等价性

- 对任何 NFA N ，存在定义同一语言的 DFA D
- 对任何 DFA D ，存在定义同一语言的 NFA N

DFA和NFA的等价性

➤ DFA和NFA可以识别相同的语言



状态1: 串以 a 结尾

状态2: 串以 ab 结尾

状态3: 串以 abb 结尾

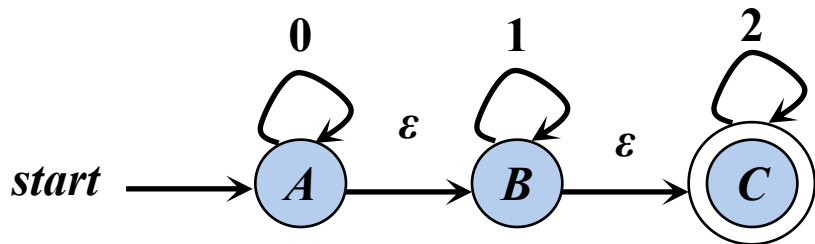
$r = (a|b)^*abb$

正则文法 $\Leftrightarrow$ 正则表达式 $\Leftrightarrow$ FA

带有“ ε -边”的NFA

$$M = (S, \Sigma, \delta, s_0, F)$$

- S : 有穷状态集
- Σ : 输入符号集合, 即输入字母表。假设 ε 不是 Σ 中的元素
- δ : 将 $S \times (\Sigma \cup \{\varepsilon\})$ 映射到 2^S 的转换函数。 $\forall s \in S, a \in \Sigma \cup \{\varepsilon\}, \delta(s, a)$ 表示从状态 s 出发, 沿着标记为 a 的边所能到达的状态集合
- s_0 : 开始状态 (或初始状态), $s_0 \in S$
- F : 接收状态 (或终止状态) 集合, $F \subseteq S$

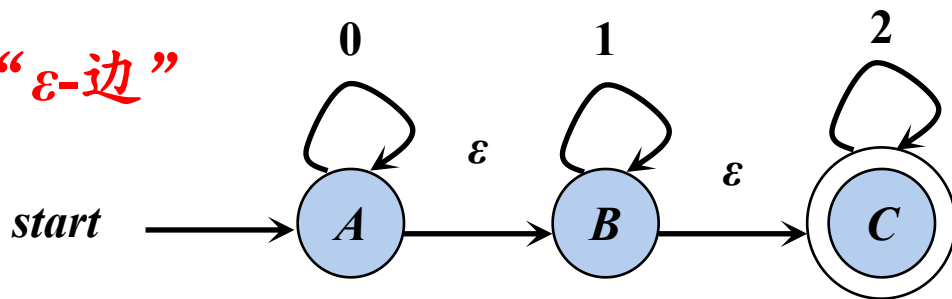


$$r = 0^*1^*2^*$$

带有和不带有“ ε -边”的NFA的等价性

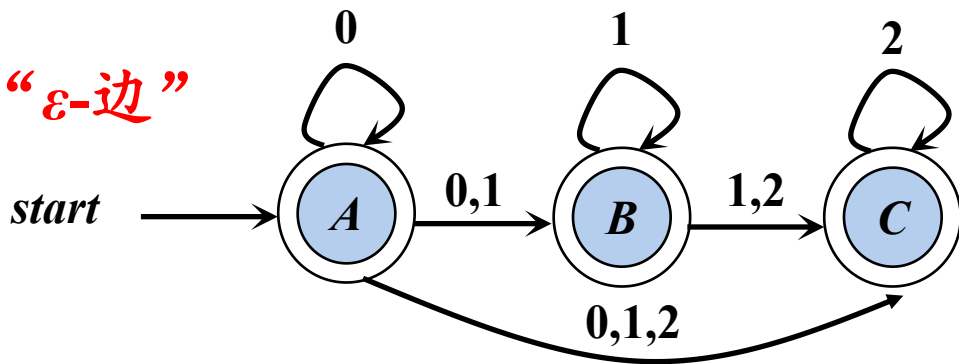
➤ 例

带“ ε -边”



$$r = 0^*1^*2^*$$

不带“ ε -边”



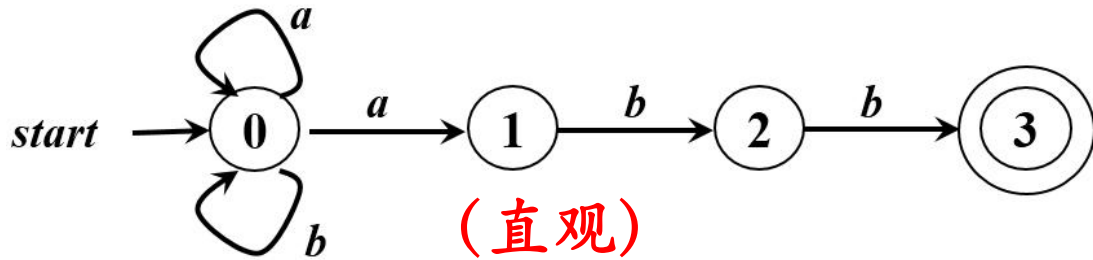
DFA的算法实现

- 输入：以文件结束符eof结尾的字符串 x 。DFA D 的开始状态 s_0 ，接受状态集 F ，转换函数 $move$ 。
- 输出：如果 D 接受 x ，则回答“yes”，否则回答“no”。
- 方法：将下述算法应用于输入串 x 。

```
 $s = s_0;$   
 $c = nextChar ();$   
while ( $c \neq eof$ ) {  
     $s = move(s, c);$   
     $c = nextChar ();$   
}  
if ( $s$ 在 $F$ 中) return “yes”;  
else return “no”;
```

- 函数 $nextChar()$ 返回输入串 x 的下一个符号
- 函数 $move(s, c)$ 表示从状态 s 出发，沿着标记为 c 的边所能到达的状态

3.2.3 从正则表达式到无穷自动机



(直观)

NFA

单词的描述

RE

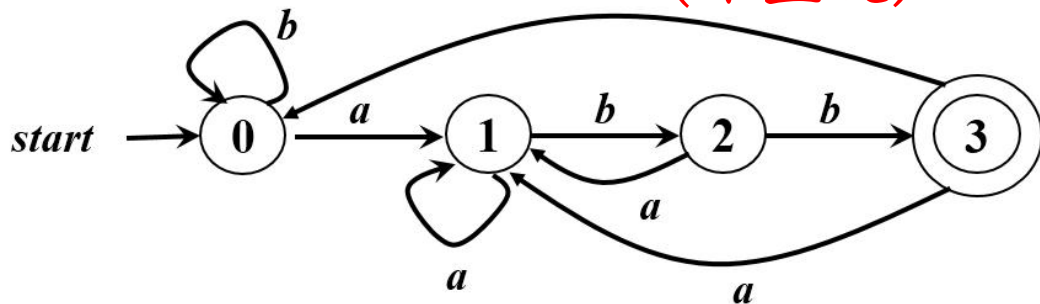
(直观)

$r = (a|b)^*abb$

单词的识别

DFA

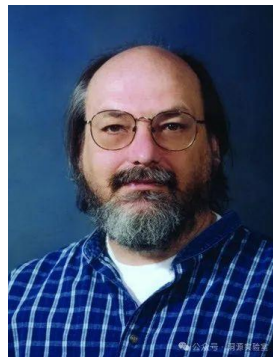
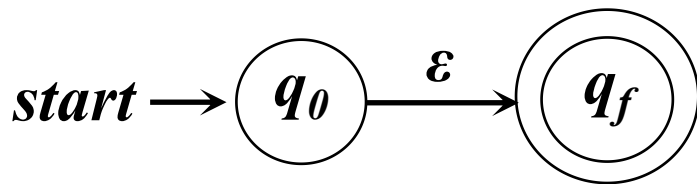
b (不直观)



根据 RE 构造 NFA

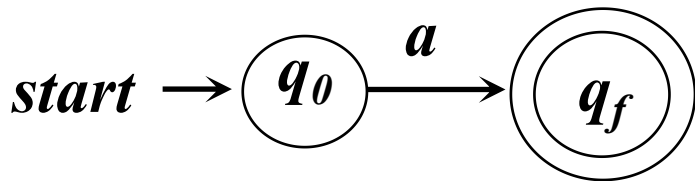
Ken Thompson(1968). "Regular expression search algorithm
(正则表达式搜索算法)"

➤ ϵ 对应的 NFA



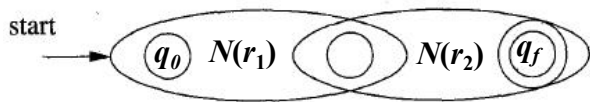
1983年图灵奖得主
肯·汤普森

➤ 字母表 Σ 中符号 a 对应的 NFA

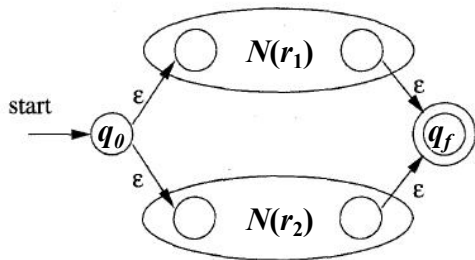


➤ 假设正则表达式 r_1 和 r_2 对应的 NFA 分别为 $N(r_1)$ 和 $N(r_2)$

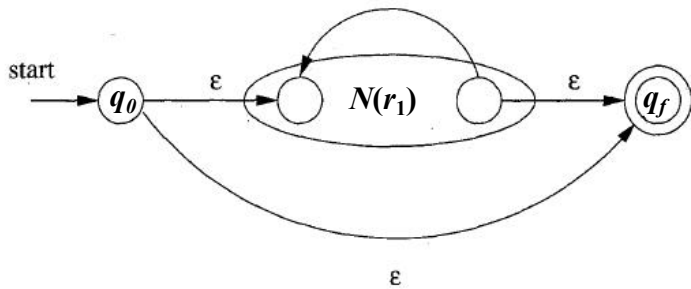
➤ $r = r_1 r_2$ 对应的 NFA



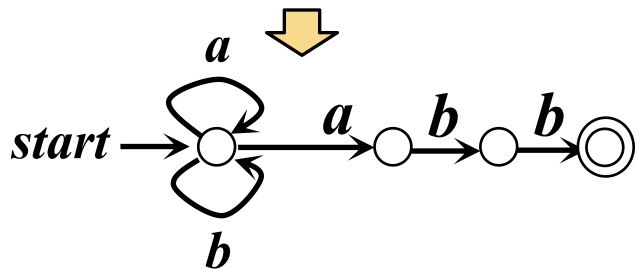
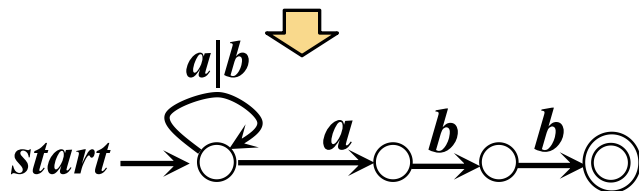
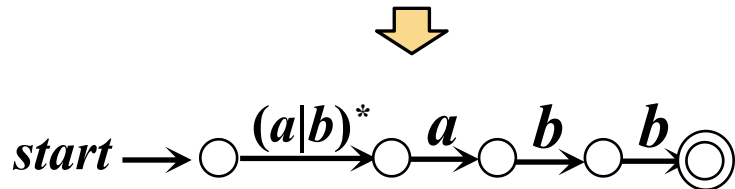
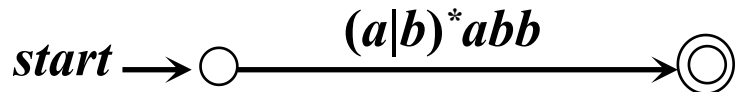
➤ $r = r_1 | r_2$ 对应的 NFA



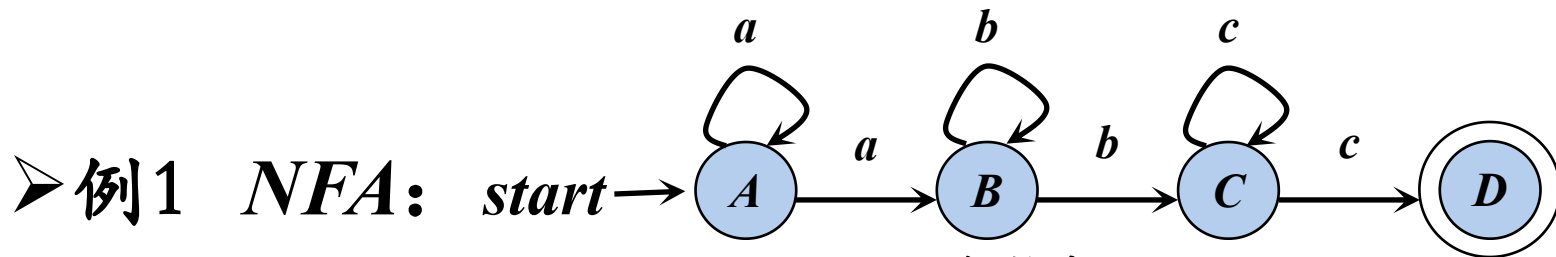
➤ $r = (r_1)^*$ 对应的 NFA



例： $r = (a|b)^*abb$ 对应的 *NFA*



从NFA到DFA的转换

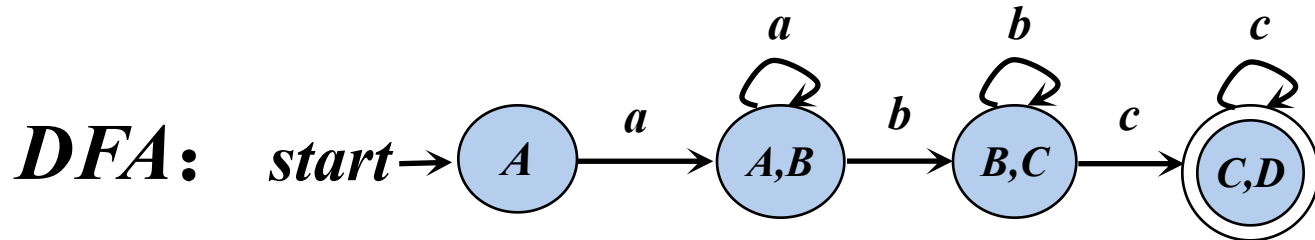


转换表

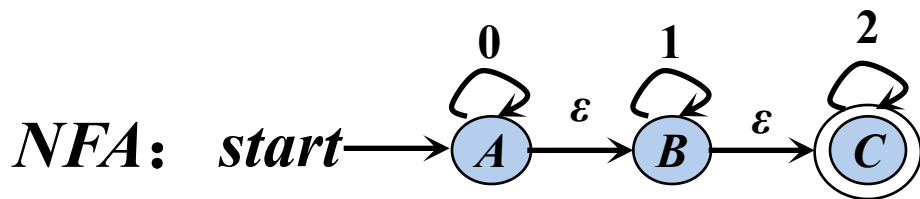
状态 \ 输入	a	b	c
A	{A,B}	$\emptyset$	$\emptyset$
B	$\emptyset$	{B,C}	$\emptyset$
C	$\emptyset$	$\emptyset$	{C,D}
D •	$\emptyset$	$\emptyset$	$\emptyset$

DFA的每个状态都是一个由NFA中的状态构成的集合,即NFA状态集合的一个子集

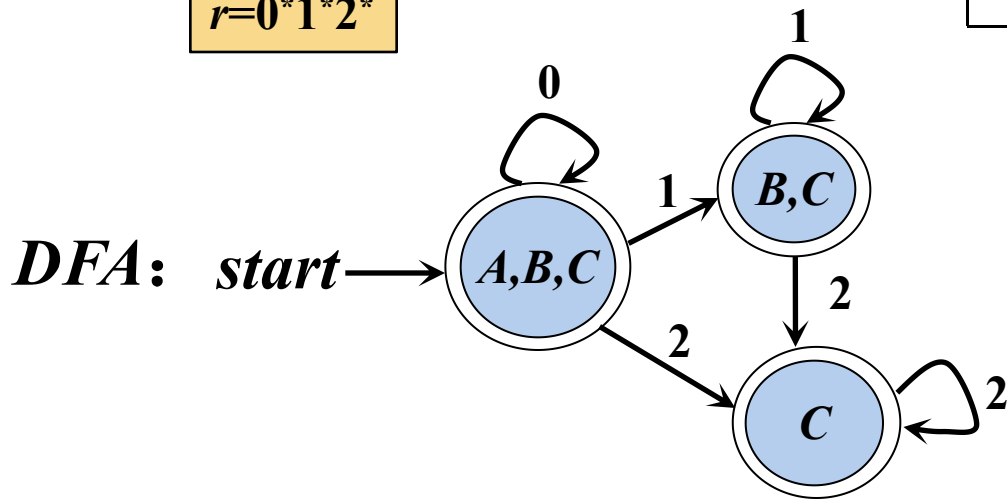
$r = aa^*bb^*cc^*$



例2：从带有 ε -边的 NFA 到 DFA 的转换



$r=0^*1^*2^*$



转换表

状态 \ 输入	0	1	2
A	{A,B,C}	{B,C}	{C}
B	$\emptyset$	{B,C}	{C}
C •	$\emptyset$	$\emptyset$	{C}

子集构造法 (subset construction)

- 输入: $NFA\ N$
- 输出: 接收同样语言的 $DFA\ D$
- 方法: 一开始, $\varepsilon\text{-closure}(s_0)$ 是 $Dstates$ 中的唯一状态, 且它未加标记;
while (在 $Dstates$ 中有一个未标记状态 T) {
 给 T 加上标记;
 for (每个输入符号 a) {
 $U = \varepsilon\text{-closure}(\text{move}(T, a))$;
 if (U 不在 $Dstates$ 中)
 将 U 加入到 $Dstates$ 中, 且不加标记;
 $Dtran[T, a] = U$;
 }
}

操作	描述
$\varepsilon\text{-closure}(s)$	能够从 NFA 的状态 s 开始只通过 ε 转换到达的 NFA 状态集合
$\varepsilon\text{-closure}(T)$	能够从 T 中的某个 NFA 状态 s 开始只通过 ε 转换到达的 NFA 状态集合, 即 $U_{s \in T} \varepsilon\text{-closure}(s)$
$\text{move}(T, a)$	能够从 T 中的某个状态 s 出发通过标号为 a 的转换到达的 NFA 状态的集合

计算 ε -closure (T)

将 T 的所有状态压入 $stack$ 中；

将 ε -closure (T)初始化为 T ；

while ($stack$ 非空) {

 将栈顶元素 t 给弹出栈中；

 for (每个满足如下条件的 u ：从 t 出发有一个标号为 ε 的转换到达状态 u)

 if (u 不在 ε -closure (T)中) {

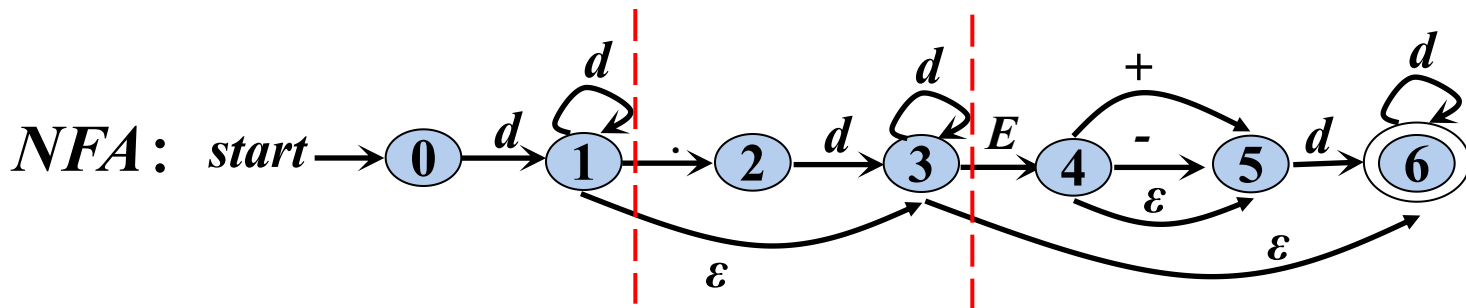
 将 u 加入到 ε -closure (T)中；

 将 u 压入栈中；

 }

}

例



➤ $digit \rightarrow 0|1|2|\dots|9$

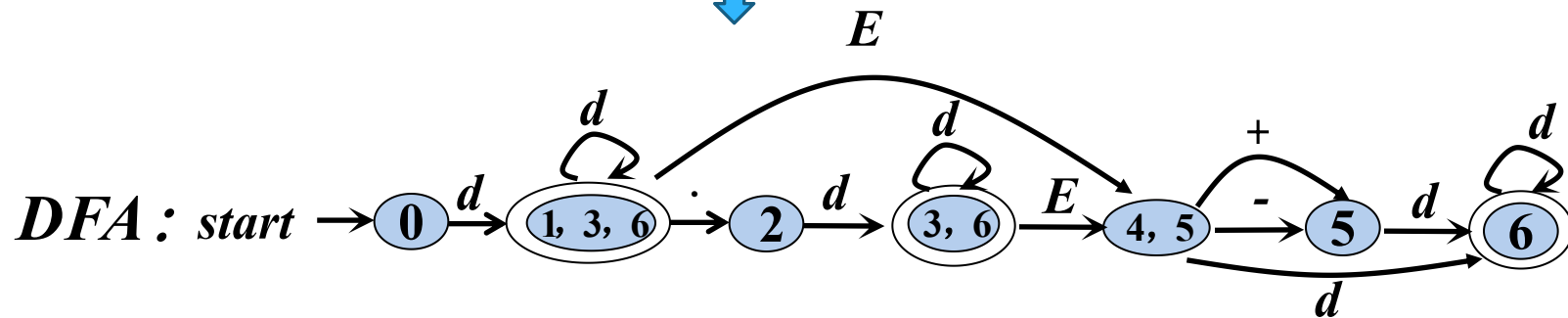
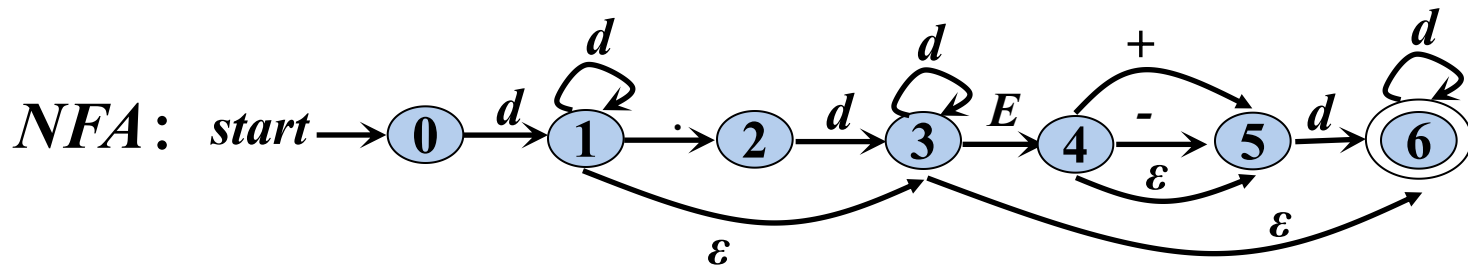
➤ $digits \rightarrow digit\ digit^*$

➤ $optionalFraction \rightarrow .digits|\epsilon$

➤ $optionalExponent \rightarrow (E(+|-|\epsilon)digits)|\epsilon$

➤ $number \rightarrow digits\ optionalFraction\ optionalExponent$

例



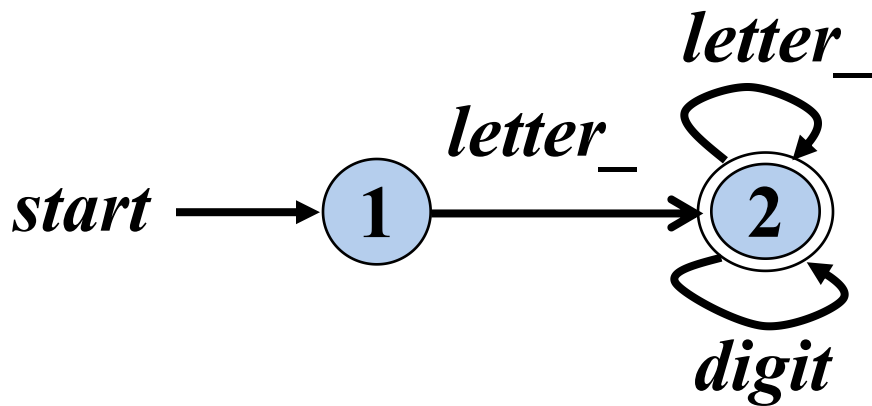
3.2.4 识别单词的 *DFA*

➤ 识别标识符的 *DFA*

digit $\rightarrow 0|1|2|\dots|9$

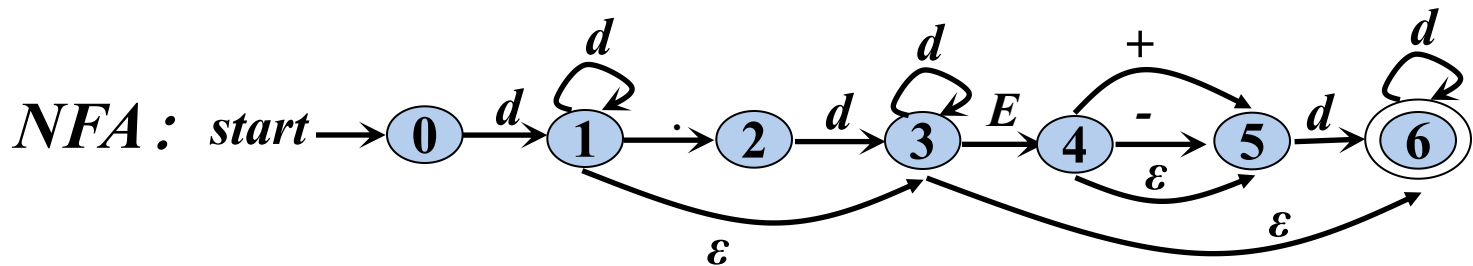
letter_ $\rightarrow A|B|\dots|Z|a|b|\dots|z|_$

id $\rightarrow \textit{letter_}(\textit{letter_}|\textit{digit})^*$

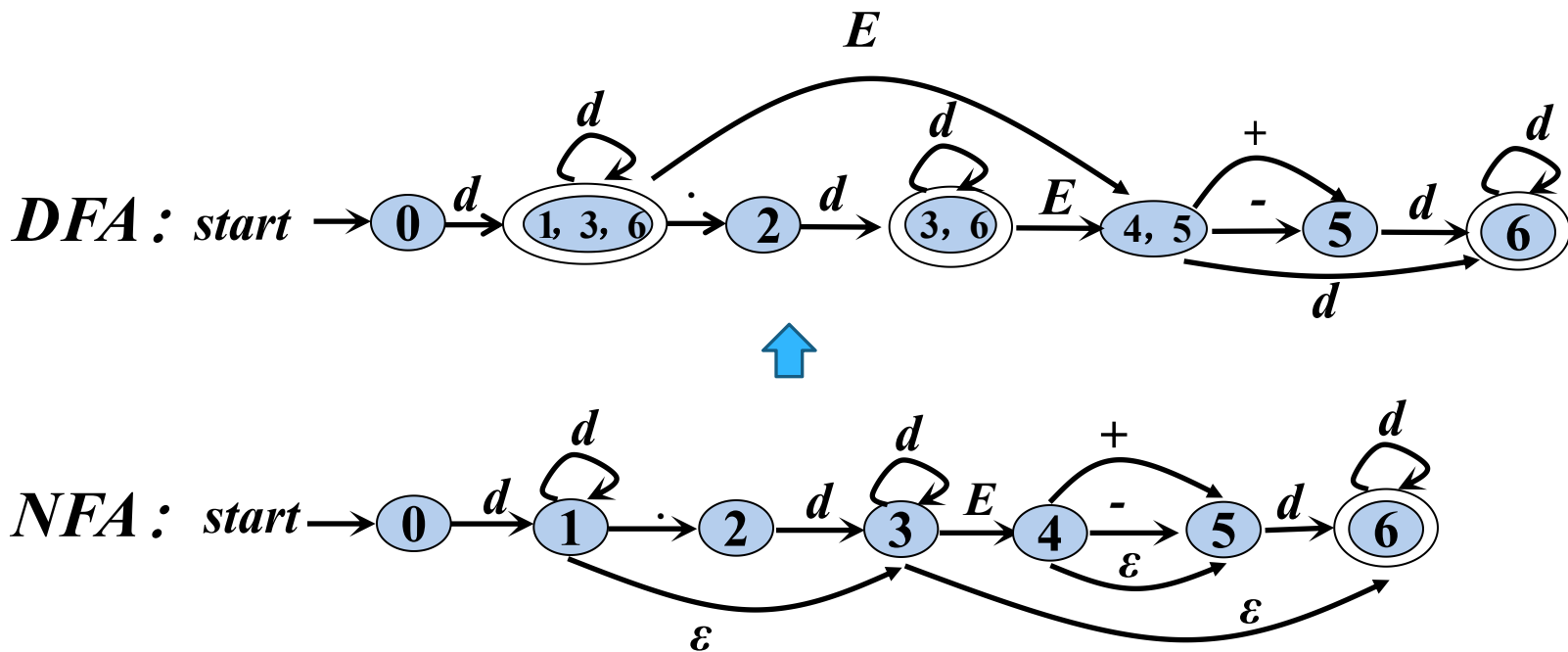


识别无符号数的 DFA

- $digit \rightarrow 0|1|2|\dots|9$
- $digits \rightarrow digit\ digit^*$
- $optionalFraction \rightarrow .digits|\epsilon$
- $optionalExponent \rightarrow (E(+|-|\epsilon)digits)|\epsilon$
- $number \rightarrow digits\ optionalFraction\ optionalExponent$

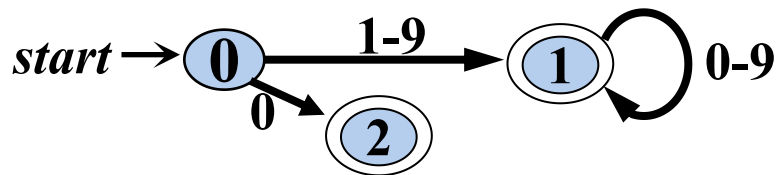


识别无符号数的 DFA

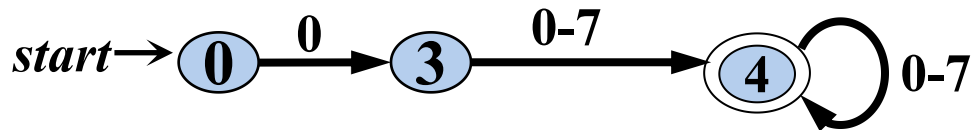


识别各进制无符号整数的 *DFA*

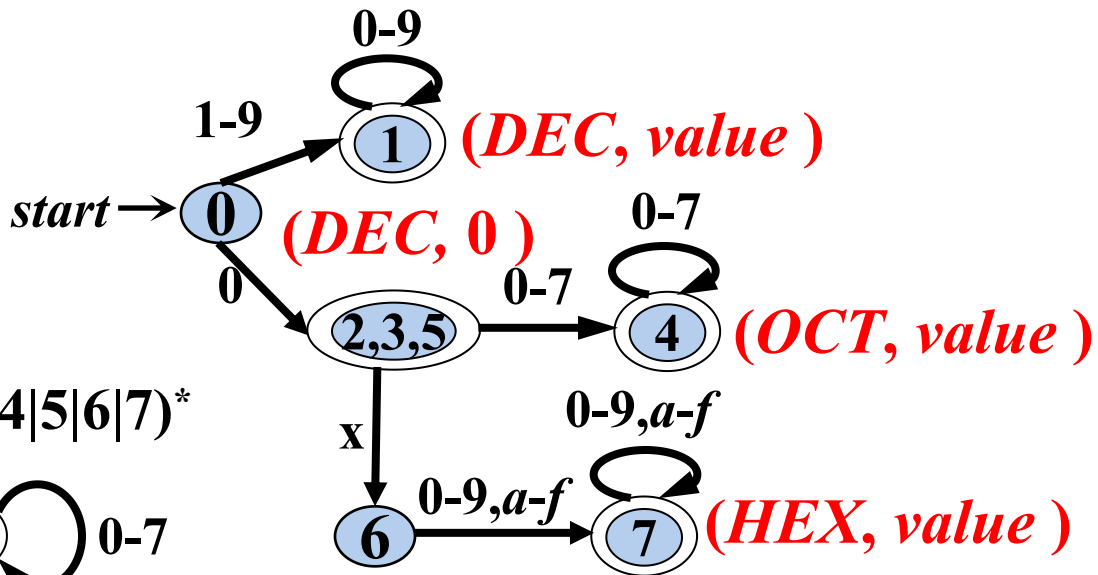
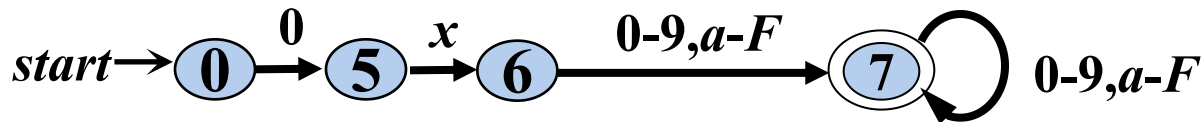
DEC $\rightarrow (1|...|9)(0|...|9)^* | 0$



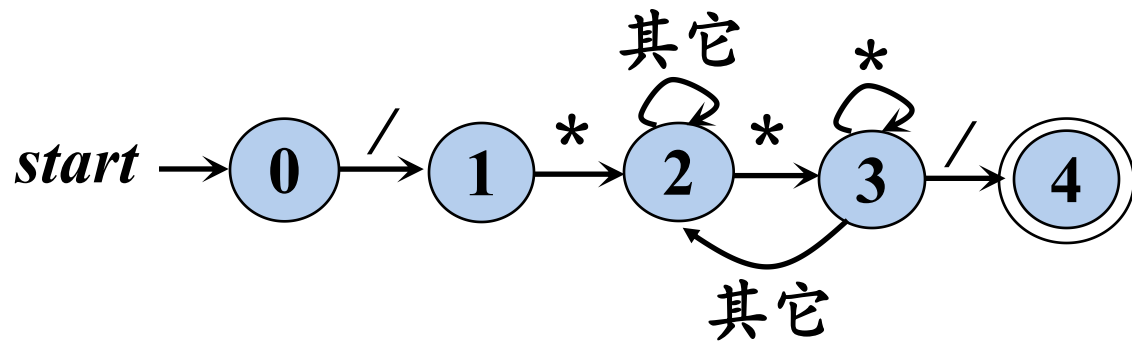
OCT $\rightarrow 0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$



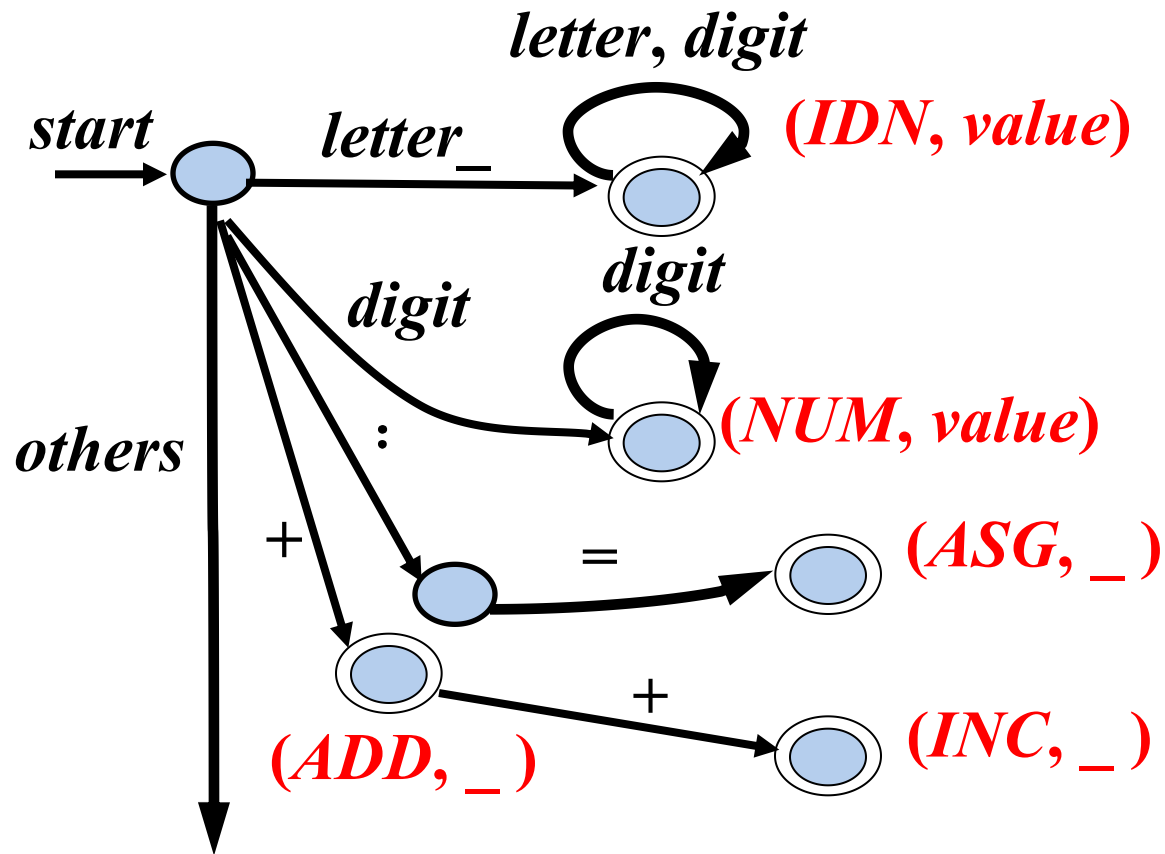
HEX $\rightarrow 0x(0|1|...|9|a|...|f|A|...|F)(0|...|9|a|...|f|A|...|F)^*$

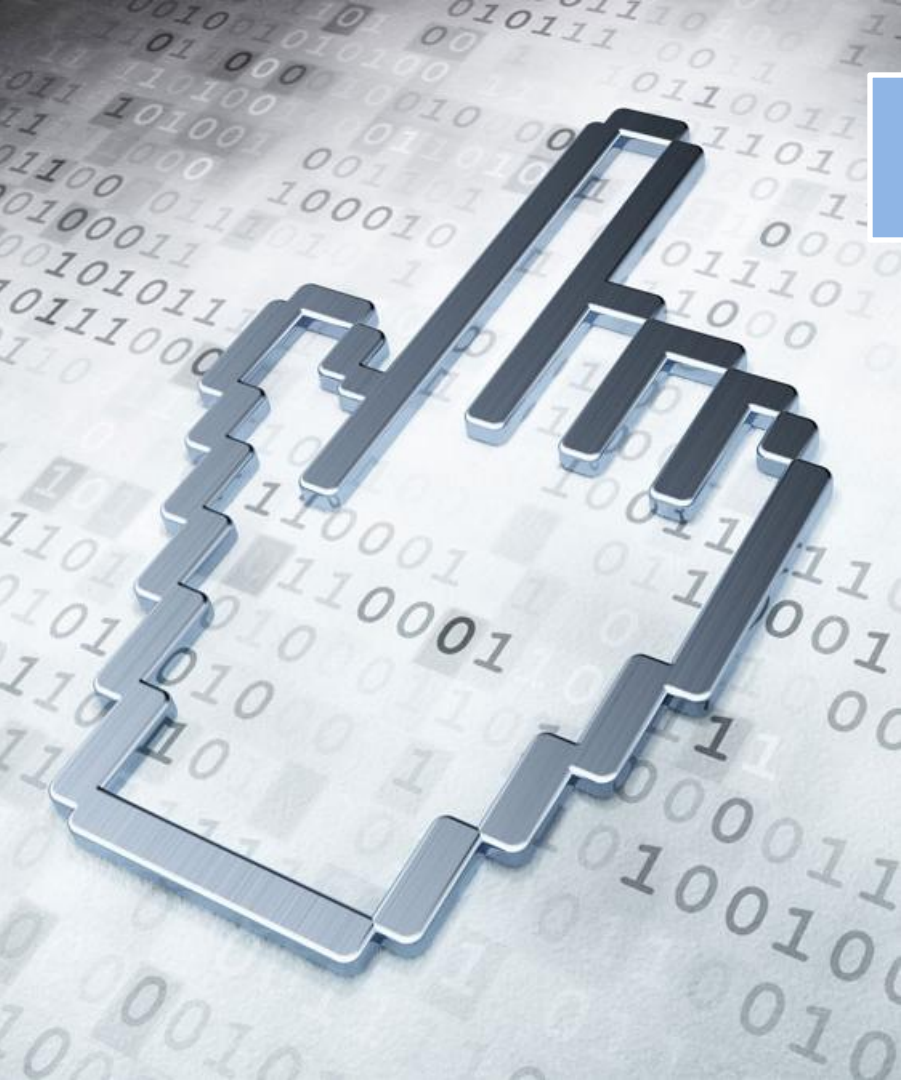


识别注释的 *DFA*



识别 *Token* 的 *DFA*





提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

3.4 词法分析阶段的错误处理

➤ 词法错误的类型

➤ 单词拼写错误

➤ 例： *int* *i* = **0x3G**; *float* *j* = **1.05e**;

➤ 非法字符

➤ 例： ~ @

➤ 词法错误检测

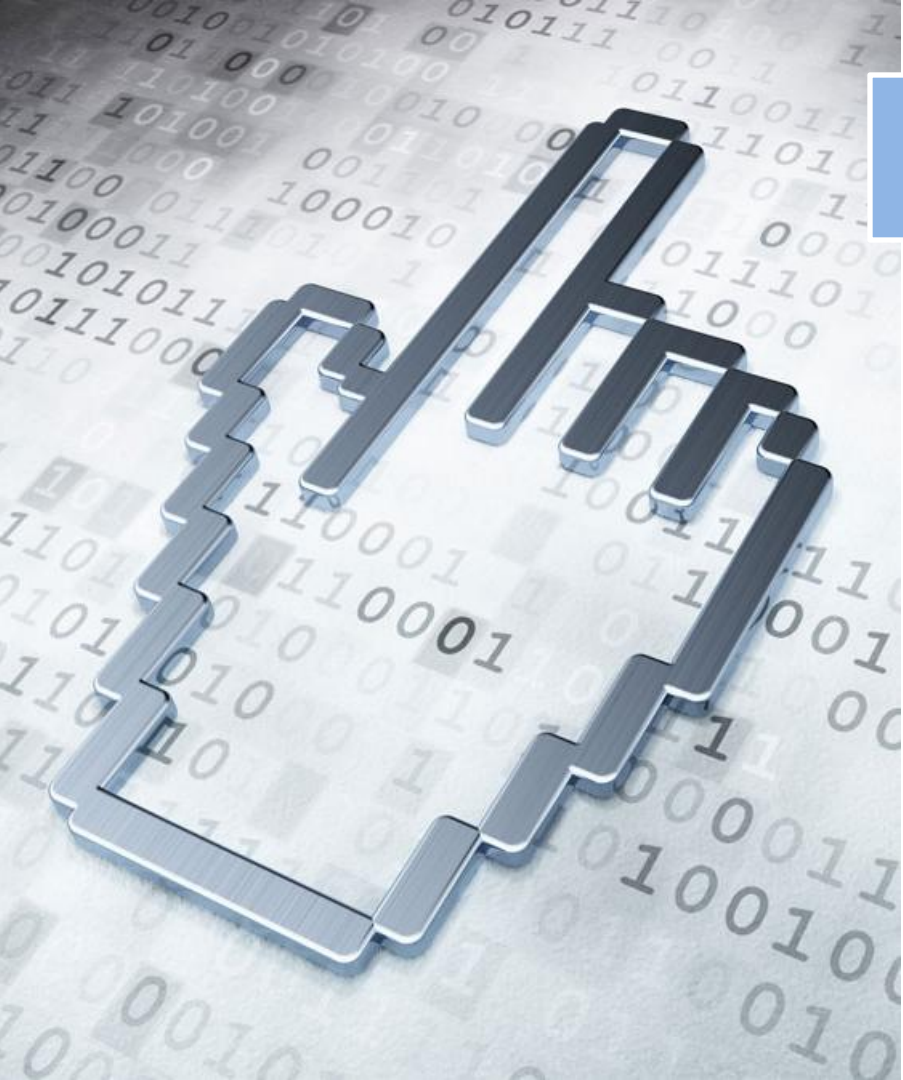
➤ 如果当前状态与当前输入符号在转换表对应项中的信息为空，而当前状态又不是**终止状态**，则调用**错误处理程序**

错误处理

- 查找已扫描字符串中最后一个对应于某终态的字符
 - 如果找到了，将该字符与其前面的字符识别成一个单词。
然后将输入指针退回到该字符，扫描器重新回到初始状态，继续识别下一个单词
 - 如果没找到，则确定出错，采用错误恢复策略

错误恢复策略

- 最简单的错误恢复策略：“恐慌模式 (*panic mode*)” 恢复
 - 从剩余的输入中不断删除字符，直到词法分析器能够在剩余输入的开头发现一个正确的字符为止



提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

3.4 词法分析器生成工具Lex

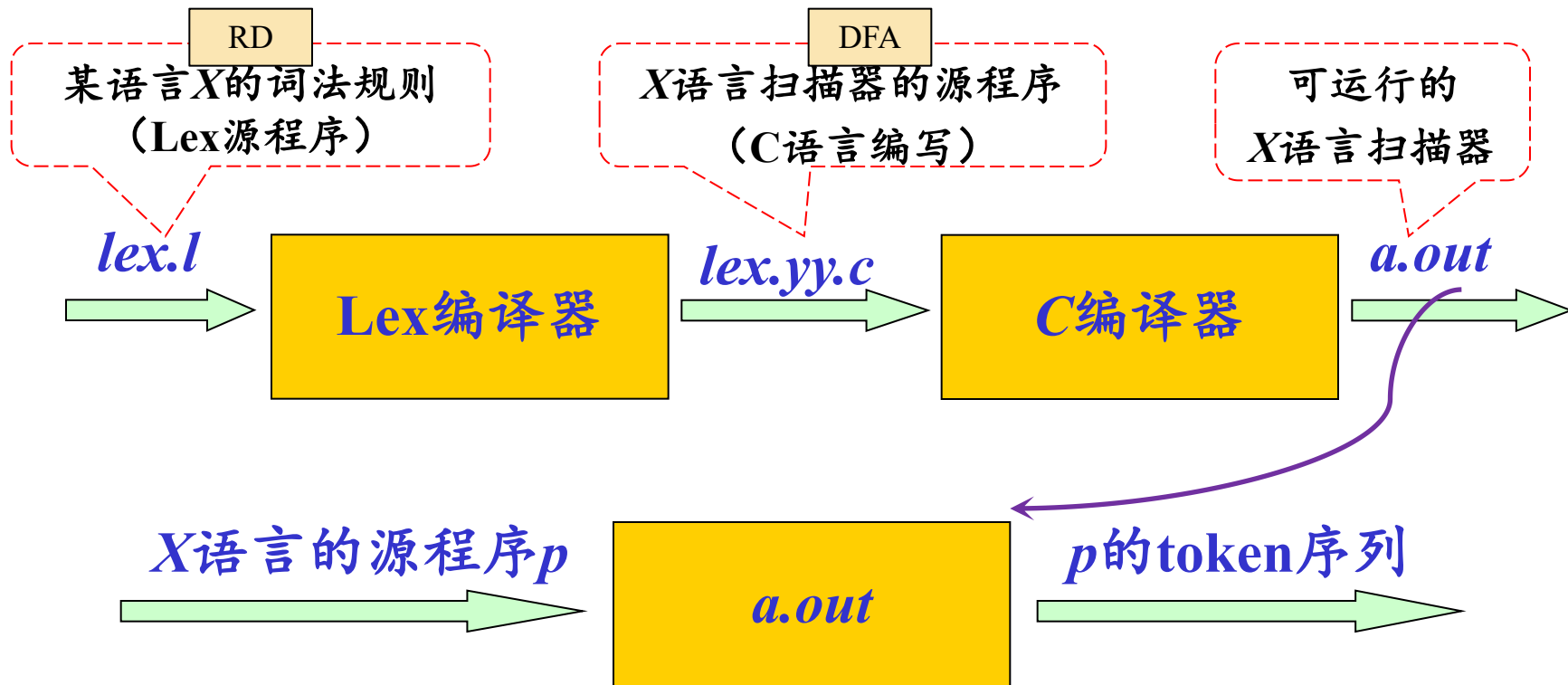
3.4 词法分析器生成工具Lex

Lesk, M.E. (October 1975). "Lex – A Lexical Analyzer Generator". *Comp. Sci. Tech. Rep. No. 39* (Murray Hill, New Jersey: Bell Laboratories).

➤ Lex的构成

- Lex语言：用于描述各类单词的**正则定义式**，及各单词被识别出来以后应执行的动作
- Lex编译器：将输入的lex源程序（RD）转化成一个词法分析器（DFA）

Lex的使用



Lex 程序的 结构

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
%}  
  
/* regular definitions */  
delim      [ \t\n]  
ws         {delim}+  
letter     [A-Za-z]  
digit      [0-9]  
id         {letter}({letter}|{digit})*  
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
```

出现在括号内的所有内容都被直接复制到文件lex.yy.c中

声明部分

```
%%  
  
{ws}      { /* no action and no return */}  
if         {return(IF);}   
then       {return(THEN);}   
else       {return(ELSE);}   
{id}      {yylval = (int) installID(); return(ID);}   
{number}  {yylval = (int) installNum(); return(NUMBER);}   
"<"       {yylval = LT; return(RELOP);}   
"<="      {yylval = LE; return(RELOP);}   
"="        {yylval = EQ; return(RELOP);}   
"<>"      {yylval = NE; return(RELOP);}   
">"       {yylval = GT; return(RELOP);}   
">="      {yylval = GE; return(RELOP);}
```

转换规则

模式 {动作}

```
%%  
  
int installID() { /* function to install the lexeme, whose  
                  first character is pointed to by yytext,  
                  and whose length is yyleng, into the  
                  symbol table and return a pointer  
                  thereto */  
}  
  
int installNum() { /* similar to installID, but puts numer-  
                   ical constants into a separate table */  
}
```

出现在辅助部分中的所有内容都被直接复制到文件lex.yy.c中

辅助过程

Lex 程序的 结构

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
%}  
  
/* regular definitions */  
delim      [ \t\n]  
ws         {delim}+  
letter     [A-Za-z]  
digit      [0-9]  
id         {letter}({letter}|{digit})*  
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
```

出现在括号内的所有内容都被直接复制到文件lex.yy.c中

声明部分

```
%%  
  
{ws}      { /* no action and no return */}  
if         {return(IF);}   
then       {return(THEN);}   
else       {return(ELSE);}   
{id}      {yylval = (int) installID(); return(ID);}   
{number}  {yylval = (int) installNum(); return(NUMBER);}   
"<"       {yylval = LT; return(RELOP);}   
"<="      {yylval = LE; return(RELOP);}   
"="        {yylval = EQ; return(RELOP);}   
"<>"      {yylval = NE; return(RELOP);}   
">"       {yylval = GT; return(RELOP);}   
">="      {yylval = GE; return(RELOP);}
```

转换规则

模式 {动作}

```
%%  
  
int installID() { /* function to install the lexeme, whose  
                  first character is pointed to by yytext,  
                  and whose length is yyleng, into the  
                  symbol table and return a pointer  
                  thereto */  
}  
  
int installNum() { /* similar to installID, but puts numer-  
                   ical constants into a separate table */  
}
```

出现在辅助部分中的所有内容都被直接复制到文件lex.yy.c中

辅助过程

扫描器自动生成的意义

- Lex的构成加快了分析器的实现速度
 - 程序员只需在很高的模式层次上描述软件，就可以依赖自动生成工具来生成详细的代码
- 修改扫描器的工作变得更加简单
 - 只需修改那些受到影响的模式，无需改写整个程序

种别码编码方案举例：龙书附录A.3

```
1) package lexer;                                // File Tag.java
2) public class Tag {
3)     public final static int
4)         AND = 256, BASIC = 257, BREAK = 258, DO = 259, ELSE = 260,
5)         EQ = 261, FALSE = 262, GE = 263, ID = 264, IF = 265,
6)         INDEX = 266, LE = 267, MINUS = 268, NE = 269, NUM = 270,
7)         OR = 271, REAL = 272, TEMP = 273, TRUE = 274, WHILE = 275;
8) }
```

- 关键字

- BASIC: int/float/char/bool
- BREAK: break
- DO: do
- WHILE: while
- IF: if
- ELSE: else
- TRUE: true
- FALSE: false

- 算符 (非单符)

- AND
- OR
- EQ: ==
- NE: !=
- GE: >=
- LE: <=

- 常量

- NUM 整型
- REAL 浮点型

- 标识符

- ID

单字符词法单元就用其自身的ASCII码作为种别码

用大于255的整数来表示非单字符词法单元的种别码

INDEX、MINUS、TEMP不是词法单元，它们将在抽象语法树中使用

本章小结

- 单词的描述
 - 正则表达式
 - 正则定义
- 单词的识别
 - 有穷自动机
 - 有穷自动机的分类
 - 从正则表达式到有穷自动机
 - 识别单词的DFA
- 词法分析阶段的错误处理
- 词法分析器生成工具Lex



结束

